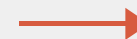




Visualization Plugins



Robot Operating System





Lecture 08: Visualization Plugins

What is Rviz Visualization plugin?

- Visualization plugins for RViz (Robot Visualization) are extensions that allow users to visualize various types of data from ROS (Robot Operating System) in 3D, such as sensor data, robot models, and planning information. These plugins can be custom-built or come as part of specific packages and enable interactive control and monitoring of robot systems. Examples include plugins for visualizing LIDAR point clouds, IMU (Inertial Measurement Unit) data, and 3D object models from a database.

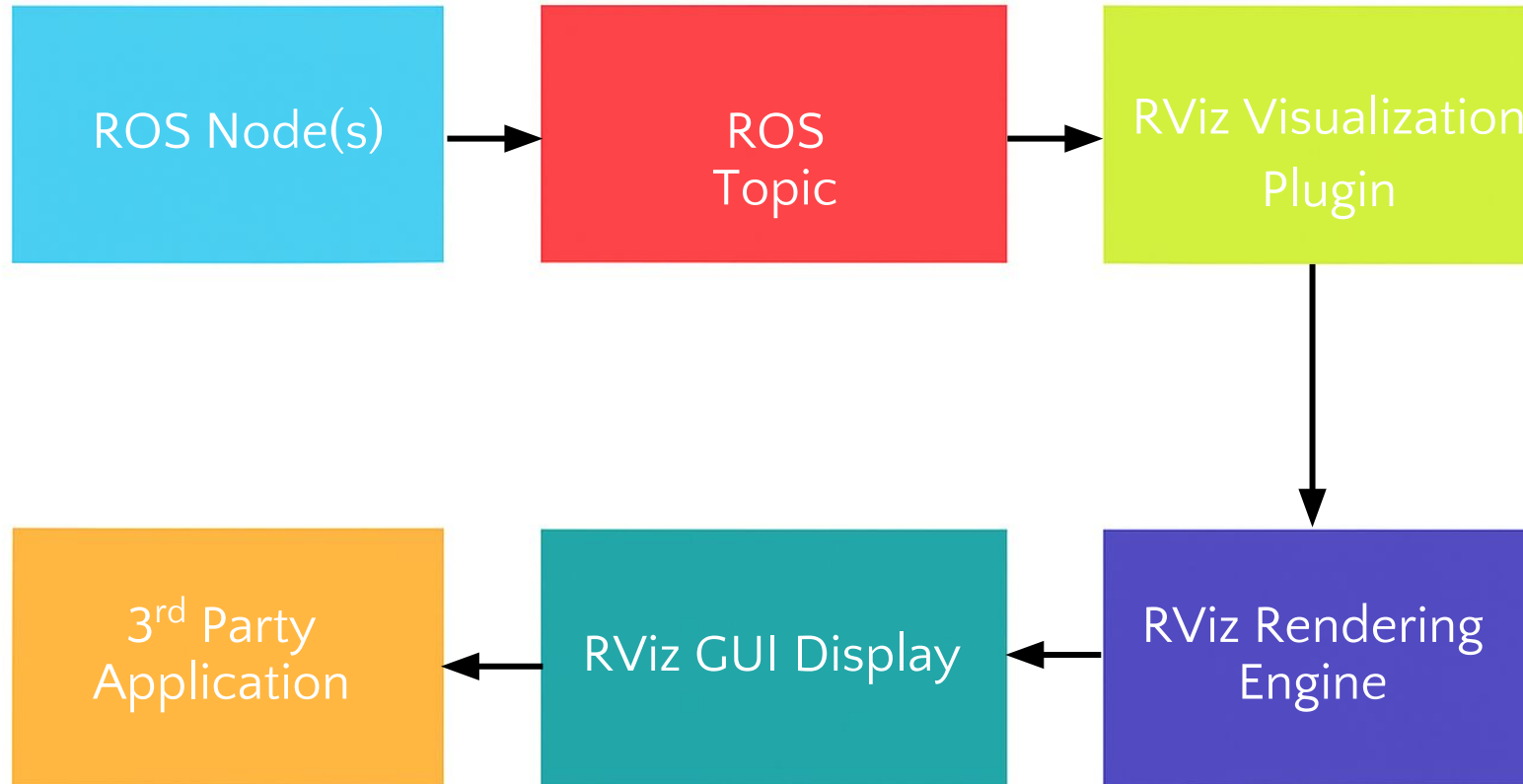
Types of RViz Plugins

Plugin Type	Usage
Display Plugin	Visualizing custom topics (e.g., heatmaps, arrows, paths)
Panel Plugin	Custom UI tools (buttons, widgets, controllers)
Tool Plugin	Interactive map editing, annotation
Visualization Marker Plugin	For arrows, cubes, meshes

Question

How do RViz Visualization Plugins work with ROS? Draw the diagram and briefly explain each block.

How RViz Visualization Plugins Work with ROS?



1. ROS Nodes

- ROS Nodes generate the data that RViz will visualize.
- They publish information such as:
 - Laser scan
 - Odometry
 - Robot pose
 - Camera images
 - Maps
- Without ROS nodes publishing data, RViz has nothing to display.

2. ROS Topics

- ROS Topics act as data channels between ROS nodes and RViz.
- Topics carry the messages that RViz plugins need:
 - /scan → Laser scans
 - /odom → Robot position
 - /map → Occupancy grid
 - /image_raw → Camera images

3. RViz Visualization Plugin

- The RViz plugin is the core component that turns ROS data into graphics.
- It subscribes to specific ROS topics (e.g., /scan, /path, /map).
- Every time a message arrives:
 - The plugin processes the data
 - Converts it into visual objects (lines, points, arrows, meshes)

4. OGRE Rendering Engine

- OGRE (Object-Oriented Graphics Rendering Engine) is the 3D graphics engine inside RViz.
- It receives the visual shapes generated by the RViz plugin.
- OGRE handles:
 - Drawing objects (points, lines, meshes)
 - Lighting and shading
 - Camera view
 - 3D scene management.
- RViz plugins do not draw directly on the screen.
- They only create render objects.
- OGRE is responsible for actually rendering those objects in 3D.

5. RViz GUI Display

- The RViz GUI is the Graphical User Interface where everything becomes visible.
- It shows the rendered 3D scene from OGRE.
- Provides user controls for:
 - Enabling/disabling plugins
 - Changing colors and sizes
 - Switching coordinate frames
 - Moving the camera view

6. 3rd-Party Application

- A 3rd-party application is any external program that publishes data into ROS.
- Examples:
 - Simulation tools (Gazebo, Webots, Unity)
 - External controllers (MATLAB/Simulink, Python scripts)
 - Custom sensor drivers or hardware interfaces
 - AI/ML modules generating predictions or maps
- The 3rd-party app publishes ROS messages just like normal ROS nodes.
- RViz plugins can visualize this data as long as:
 - The messages follow ROS formats
 - They are published on known topics

Question

How can you create and integrate a custom RViz panel plugin (such as a “ButtonPanel”) using Python, a plugin XML file, and minimal CMake configuration? Also explain How to Build and Run this Plugin?

Answer

- To create and integrate a simple RViz panel plugin, follow these three required components:
 1. Python Plugin File (button_panel.py)
 2. Plugin Description XML (button_panel.xml)
 3. CMakeLists.txt Additions

1. Python Plugin File (button_panel.py)

```
from python_qt_binding.QtWidgets import QPushButton, QVBoxLayout
import rviz
import rospy
```

```
class ButtonPanel(rviz.Panel):
    def __init__(self, parent=None):
        super(ButtonPanel, self).__init__(parent)
```

```
        layout = QVBoxLayout()
```

```
        # Create a button
```

```
        self.button = QPushButton("Click Me!")
        self.button.clicked.connect(self.on_click)
```

```
        layout.addWidget(self.button)
        self.setLayout(layout)
```

```
def on_click(self):
    rospy.loginfo("Button was clicked!")
```

- . This file defines the RViz panel.
- . It creates a button and logs a message when clicked.

2. Plugin Description XML (button_panel.xml)

```
<class_library path="lib/libbutton_panel">
  <class
    name="button_panel/ButtonPanel"
    type="button_panel.ButtonPanel"
    base_class_type="rviz.Panel">
    <description>Simple RViz panel with one button</description>
  </class>
</class_library>
```

- This file tells RViz how to load your Python class as a plugin.

3. CMakeLists.txt Additions

```
catkin_install_python(  
  PROGRAMS src/button_panel.py  
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}  
)
```

```
install(FILES button_panel.xml  
  DESTINATION ${CATKIN_PACKAGE_SHARE_DESTINATION}  
)
```

- . This file Installs the Python script and XML file.

How to Build and Run the Plugin

- 1. Build the workspace

```
catkin_make
```

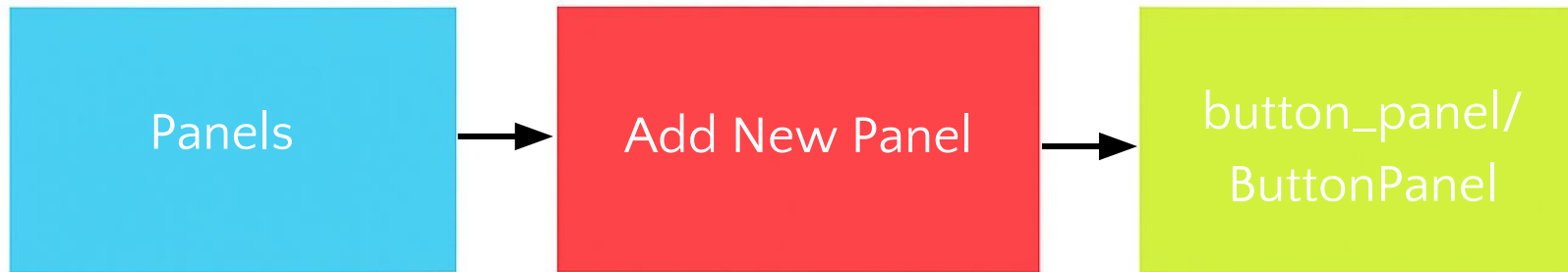
```
source devel/setup.bash
```

- 2. Launch RViz

```
rviz
```

How to Build and Run the Plugin

- 3. Add your custom panel, inside RViz:



- Clicking the button prints to your terminal:

`[INFO] Button was clicked!`

Q & A





Thank
you!